

NumPy Searching Arrays

[< Previous](#)[Next >](#)

Searching Arrays

You can search an array for a certain value, and return the indexes that get a match.

To search an array, use the `where()` method.

Example

[Get your own Python Server](#)

Find the indexes where the value is 4:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 4, 4])

x = np.where(arr == 4)

print(x)
```

[Try it Yourself »](#)

The example above will return a tuple: `(array([3,`

```
5, 6],)
```

Which means that the value 4 is present at index 3, 5, and 6.

Example

Find the indexes where the values are even:

```
import numpy as np
```

[Tutorials ▼](#)[References ▼](#)[Exercises ▼](#)[Certificates ▼](#)[In](#)[HTML](#)[CSS](#)[JAVASCRIPT](#)[SQL](#)[PYTHON](#)[JAVA](#)[PHP](#)[HOW TO](#)[W3.CSS](#)[C](#)[C](#)

NumPy Tutorial

[NumPy HOME](#)[NumPy Intro](#)[NumPy Getting Started](#)[NumPy Creating Arrays](#)[NumPy Array Indexing](#)[NumPy Array Slicing](#)[NumPy Data Types](#)[NumPy Copy vs View](#)[NumPy Array Shape](#)[NumPy Array Reshape](#)[NumPy Array Iterating](#)[NumPy Array Join](#)[NumPy Array Split](#)[NumPy Array Search](#)[NumPy Array Sort](#)[NumPy Array Filter](#)

NumPy Random

[Random Intro](#)[Data Distribution](#)[Random Permutation](#)[Seaborn Module](#)[Normal Distribution](#)[Binomial Distribution](#)[Poisson Distribution](#)

```
print(x)
```

[Try it Yourself »](#)

Example

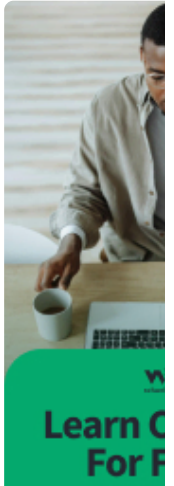
Find the indexes where the values are odd:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])

x = np.where(arr%2 == 1)

print(x)
```

[Try it Yourself »](#)

PICKER



ADVERTISEMENT

Uniform Distribution
Logistic Distribution
Multinomial Distribution
Exponential Distribution
Chi Square Distribution
Rayleigh Distribution

[REMOVE ADS](#)

Search Sorted

There is a method called `searchsorted()` which performs a binary search in the array, and returns the index where the specified value would be inserted to maintain the search order.

The `searchsorted()` method is assumed to be used on sorted arrays.

Example

Find the indexes where the value 7 should be inserted:

```
import numpy as np

arr = np.array([6, 7, 8, 9])

x = np.searchsorted(arr, 7)

print(x)
```

[Try it Yourself »](#)

Example explained: The number 7 should be inserted on index 1 to remain the sort order.

The method starts the search from the left and returns the first index where the number 7 is no longer larger than the next value.

Search From the Right Side

By default the left most index is returned, but we can give `side='right'` to return the right most index instead.

Example

Find the indexes where the value 7 should be inserted, starting from the right:

```
import numpy as np

arr = np.array([6, 7, 8, 9])

x = np.searchsorted(arr, 7, side='right')

print(x)
```

Try it Yourself »

Example explained: The number 7 should be inserted on index 2 to remain the sort order.

The method starts the search from the right and returns the first index where the number 7 is no longer less than the next value.

Multiple Values

To search for more than one value, use an array with the specified values.

Example

Find the indexes where the values 2, 4, and 6 should be inserted:

```
import numpy as np

arr = np.array([1, 3, 5, 7])

x = np.searchsorted(arr, [2, 4, 6])

print(x)
```

Try it Yourself »

The return value is an array: `[1 2 3]` containing the three indexes where 2, 4, 6 would be inserted in the original array to maintain the order.

Exercise ?

Consider the following code:

```
import numpy as np
arr = np.array([15, 38, 41, 46])
x = np.where(arr%2 == 1)
print(x)
```

What will be the printed result?

- ☐ (array([1, 3]),)
- ☐ (array([15, 41]),)
- ☐ (array([0, 2]),)
- ☐ (array([38, 46]),)